Performance Metrics

How well is your model working?

Thinking about probabilities

A classifier obtained the following results:

	True_class(x)	Pred(x)
x1	0	0
x2	0	1
x3	0	1
x4	0	0
x5	0	0
x6	0	0
x7	0	0
x8	1	1
x9	1	0
x10	1	1

Thinking about probabilities

	True_class(x)	Pred(x)
x1	0	0
x2	0	1
x3	0	1
x4	0	0
x5	0	0
x6	0	0
x7	0	0
x8	1	1
x9	1	0
x10	1	1

We are going to make the following bet an infinite number of times:

Choose one of x1,…,x10 randomly.

If the prediction made by the model for that example is correct (that is, True_class(x)==Pred(x)), you get $1, otherwise you get nothing.

But you have to play some amount of money to play.

Should you play if the price to play is 1 cent?

Should you play if the price to play is 90 cents?

How much should you pay to play every time and expect to make a profit?

Accuracy

- Fraction (or percentage) of correct predictions made by a classifier
- Equivalently, probability that a prediction will be correct

acc = correct predictions/total predictions

= 1 - incorrect predictions/total predictions

Advantage:

It can be used for two-class (positive and negative) and multiple class problems

Disadvantage:

Uninformative when classes are unbalanced. Always predicting the majority class yields high accuracy but it’s meaningless

Thinking about probabilities

	True_class(x)	Pred(x)
x1	0	0
x2	0	1
x3	0	1
x4	0	0
x5	0	0
x6	0	0
x7	0	0
x8	1	1
x9	1	0
x10	1	1

We are going to make the following bet an infinite number of times:

Choose a positive example – an example whose true class is 1 (that is, one of x8,X9,x10) randomly.

If the prediction made by the model is correct (that is, True_class(x)==Pred(x)), you get $1, otherwise you get nothing.

But you have to play some amount of money to play.

Should you play if the price to play is 1 cent?

Should you play if the price to play is 90 cents?

How much should you pay to play every time and expect to make a profit?

Recall

- Fraction (or percentage) of correct predictions made by a classifier ON EXAMPLES THAT BELONG TO THE POSITIVE CLASS

Thinking about probabilities

	True_class(x)	Pred(x)
x1	0	0
x2	0	1
x3	0	1
x4	0	0
x5	0	0
x6	0	0
x7	0	0
x8	1	1
x9	1	0
x10	1	1

We are going to make the following bet an infinite number of times:

Choose an example with a positive prediction – an example whose predicted class is 1 (that is, one of x2,x3,x8,x10) randomly.

If the prediction made by the model is correct (that is, True_class(x)==Pred(x)), you get $1, otherwise you get nothing.

But you have to play some amount of money to play.

Should you play if the price to play is 1 cent?

Should you play if the price to play is 90 cents?

How much should you pay to play every time and expect to make a profit?

Precision

- Fraction (or percentage) of correct POSITIVE predictions made by a classifier

Confusion Matrix for binary classification (positive vs. negative)

Actual \ Prediction	0	1
0	TN	FP
1	FN	TP

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \times precision \times recall}{precision + recall}$$

$$accuracy = \frac{TP + TN}{TP + FN + TN + FP}$$

$$specificity = \frac{TN}{TN + FP}$$

Confusion Matrix
2D array of counters describing the behavior of a classifier

Confusion matrix for a (very good) classifier on the MNIST dataset

		Predicted Labels									
		0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
True Labels	**0**	987	1	2	0	0	0	2	0	7	1
	1	0	977	7	2	3	2	0	2	6	1
	2	2	3	976	4	4	0	1	4	6	0
	3	0	1	18	951	0	14	0	3	9	4
	4	0	1	2	0	979	0	2	0	3	13
	5	3	0	3	9	5	968	2	0	5	5
	6	1	3	2	0	0	7	982	0	5	0
	7	3	4	3	0	13	0	0	969	0	8
	8	2	6	4	7	3	5	2	3	966	2
	9	1	1	2	6	12	2	0	8	5	963

Performance metrics for regression

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

Mean squared error

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|Y_i - \hat{Y}_i|$$

Mean absolute error

Exercise

Suppose we have the true binary class *y* of a set of examples and the prediction *p* made a classifier, where *p* and *y* are 1D arrays.

Write the following functions to evaluate the performance the classifier:

- *accuracy(y,p)*
- *precision(y,p)*
- *recall(y,p)*
- *f1(y,p)*
- *specificity(y,p)*